SRI LANKA INSTITUTE OF INFORMATION TECHNOLOGY

BSc. (Hons) in Computer Systems Engineering



IE2074 - Control Theory
Final Assignment

Design and Implementation of PID-Based DC Motor Speed Control using ESP32 and L298N Motor Driver

Control Theory – IE2074

Group 14

Assignment Title	PID-Based DC Motor Speed Control
Student ID and Name	IT24103485 - B.A.Sellapperuma, IT24103092 - R.M.V.S.Amarasinghe

1. Introduction

Speed control of DC motors sits right at the heart of modern control engineering. You'll find everywhere industrial automation, robotics, electric vehicles, conveyor systems, you name it. And here's the thing: without a well-designed control strategy, DC motors are inherently messy. Load changes, voltage fluctuations, mechanical friction, any of these can knock the motor off its target speed.

So, what's the solution? A Proportional-Integral-Derivative (PID) controller. This report walks through the complete design and implementation of a PID-based closed-loop speed controller for a DC motor. The hardware backbone is an ESP32 microcontroller driving an L298N H-Bridge motor driver via PWM, with an N20 Gear Motor's Hall sensor encoder feeding real-time speed data back into the control loop.

Why PID? Because it works. It's the most widely deployed feedback control strategy in industry and for good reasons. The three-term structure covers your bases: proportional action for an immediate correction, integral action to hunt down and eliminate steady-state error, and derivative action to dampen oscillations before they get out of hand.

The objectives of this assignment were to:

- Understand the theoretical foundation of PID control and its discrete-time mathematical formulation
- Configure PWM-based motor control using the ESP32's LEDC peripheral
- Interface the L298N motor driver for bidirectional motor operation
- Implement a complete closed-loop control system in Arduino C/C++
- Systematically tune Kp, Ki, and Kd and analyze the step response
- Evaluate key performance metrics — rise time, settling time, overshoot, and steady-state error

2. Design Methodology

2.1 Working Principle of PID Control

A PID controller is a feedback mechanism that constantly watches the gap between where you want to be (setpoint) and where you are (process variable), then fires a corrective output made up of three terms. The continuous-time control law is:

$$\mathbf{u(t) = K_p \cdot e(t) + K_i \cdot \int e(t)\,dt + K_d \cdot d(e(t))/dt}$$

Where:

- u(t) = Control output (PWM duty cycle applied to the motor driver)
- e(t) = Error signal = Setpoint speed (r) - Measured speed (y)
- Kp = Proportional gain scales the response proportional to the current error
- Ki = Integral gain eliminates accumulated steady-state error over time
- Kd = Derivative gain predicts future error based on rate of change
- t = Time variable

Where: u(t) is the control output (PWM duty cycle), e(t) is the error setpoint minus measured speed, Kp is proportional gain, Ki is integral gain, Kd is derivative gain, and t is time.

2.2 Effect of Each PID Term

Term	Function	Too High Effect	Too Low Effect
Proportional (Kp)	Reduces error proportionally; immediate corrective action	Oscillations, overshoot, instability	Slow response, large steady-state error
Integral (Ki)	Eliminates steady-state error by accumulating error over time	Integral windup, slow damping, oscillation	Persistent steady-state error
Derivative (Kd)	Dampens oscillations by reacting to rate of error change	Noise amplification, jerky output	Overshoot, underdamped response

2.3 Closed-Loop System Architecture

The system is a classic feedback loop: reference speed → error node → PID controller → PWM generator → L298N H-Bridge → N20 motor → Hall encoder → back to the error node. The Hall sensor encoder counts pulses over a fixed 100 ms window, and the ESP32 converts that count to RPM using:

Speed (RPM) = (Pulse Count / Pulses per Revolution) × (60 / dt)

Hardware interrupt service routines (ISRs) on GPIO 5 handle pulse counting no polling, no missed edges, just clean real-time feedback on every rising edge from encoder channel A.

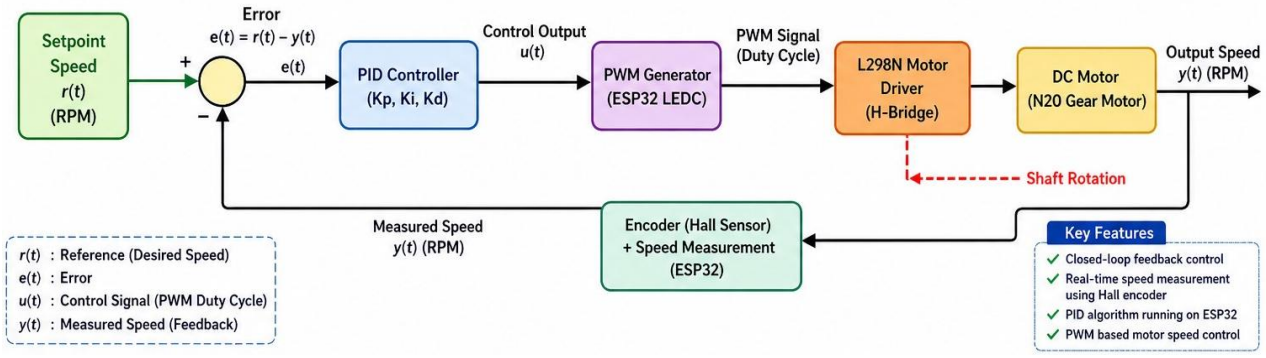
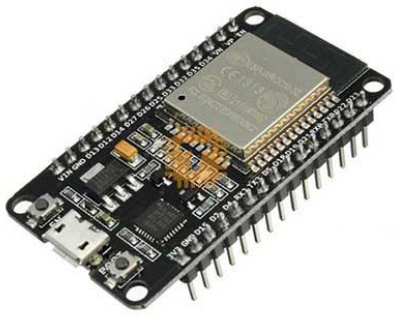


Figure : Closed-Loop PID Control System Block Diagram (N20 Gear Motor Plant)

3. Component Selection

3.1 ESP32 Development Board

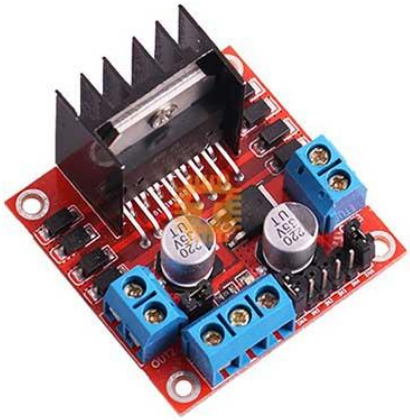
The ESP32 is a dual-core 32-bit microcontroller running up to 240 MHz, and it's frankly overkill for this job but that's a good thing. Its LEDC (LED Control) PWM peripheral supports up to 16 independent channels with 8–16-bit resolution at configurable frequencies; we used 5 kHz to push switching noise above the audible range while keeping resolution high. Any GPIO can be configured as a hardware interrupt, which is exactly what we needed for the encoder. And its 3.3V logic talks directly to the L298N without any level-shifting drama.



3.2 L298N Motor Driver Module

The L298N is a dual H-Bridge IC that takes the ESP32's 3.3V PWM logic and translates it into the current and voltage a real motor needs — up to 2A per channel, up to 46V. Speed is controlled via the ENA pin (PWM in), direction via IN1 and IN2. The onboard 5V regulator also means the 9V battery feeds the motor while keeping logic levels clean. Back-EMF protection diodes are already baked in, so we didn't need to add any externally.

Key connections: GPIO18 → ENA (PWM speed), GPIO19 → IN1 (Forward), GPIO21 → IN2 (Reverse), OUT1/OUT2 → Motor terminals, VCC → 9V battery, GND shared with ESP32.



3.3 N20 Gear Motor with Hall Sensor Encoder

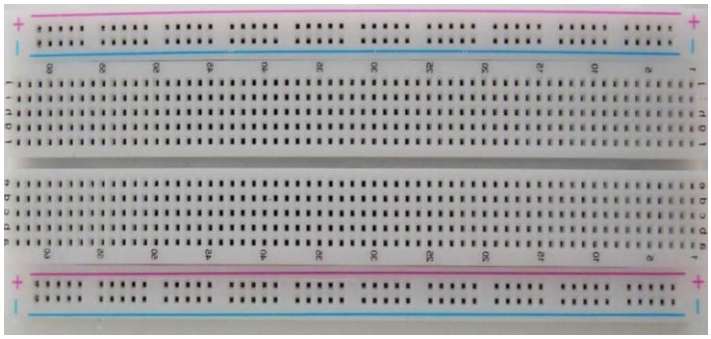
The N20 Gear Motor (6V, 300RPM) is a compact brushed DC gear motor with a built-in quadrature Hall sensor encoder. That encoder is the key — it gives us two-channel (Ch. A and Ch. B) pulse trains for both speed and direction. Channel A on GPIO 5 drives the interrupt-based pulse counter; channel B on GPIO 17 provides quadrature for direction detection. Encoder VCC is 3.3V, drawn straight from the ESP32.



3.4 Component Summary

Component	Specification	Qty	Role
ESP32 Dev Board	240 MHz, 3.3V, Dual Core	1	Microcontroller / PID host
L298N Motor Driver	2A/channel, up to 46V	1	H-Bridge motor driver
N20 Gear Motor w/ Hall Encoder	6V, 300RPM, Ch.A + Ch.B	1	Controlled plant
9V Battery	PP3 / 6F22 snap connector	1	Motor power source
Breadboard	830 tie-point	1	Prototyping platform
Connecting Wires	Jumper wires M-M / M-F	Set	Electrical connections







4. Results & Testing

4.1 Circuit Diagram and Practical Circuit

The wiring follows a straightforward topology. ESP32 GPIO18 drives the ENA PWM pin; GPIO19 and GPIO21 handle motor direction via IN1 and IN2. Encoder channel A lands on GPIO5 as a rising-edge interrupt input; channel B goes to GPIO17. The encoder runs off 3.3V from the ESP32. All grounds — battery, L298N, ESP32, encoder — are tied to a single common node to kill any floating reference issues. The ESP32 itself is powered via USB independently of the motor supply.

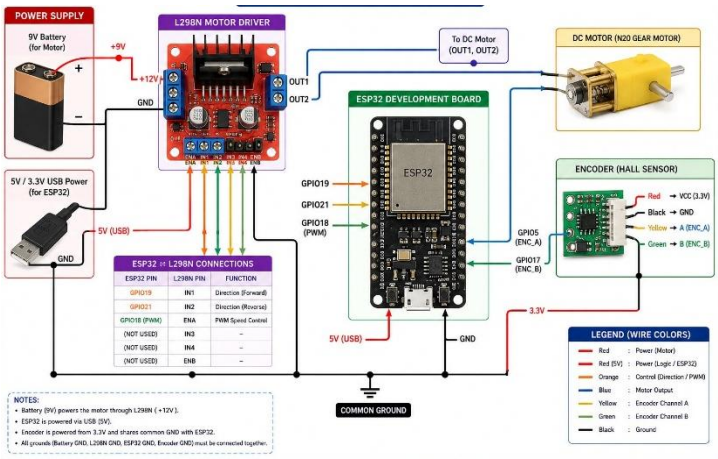


Figure 1: Circuit Wiring Diagram – ESP32, L298N & N20 Gear Motor with Hall Encoder

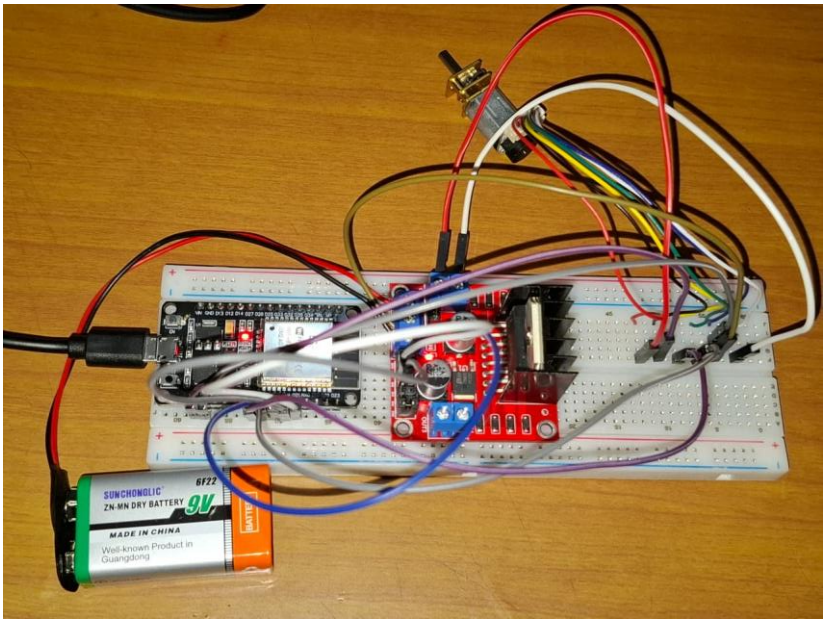


Figure 2: Practical Implementation on Breadboard – ESP32, L298N, N20 Motor & 9V Battery

4.2 Source Code

The complete firmware was written in Arduino C/C++. Key components are the encoder ISR, the PID computation function, PWM output via the LEDC peripheral, and a TCP-over-WiFi interface that lets the Python GUI send START/STOP commands and updated PID parameters in real time — no reflashing required during tuning sessions.

```
#include <WiFi.h>

// --- New Pin Definitions ---
#define IN1_PIN         19
#define IN2_PIN         21
#define ENB_PIN         18   // PWM pin
#define ENC_A_PIN       5
#define ENC_B_PIN       17

// --- Control Timing ---
#define SAMPLE_TIME_MS  20   // Faster control loop
#define LOG_INTERVAL_MS 100
#define PWM_FREQ        5000
#define PWM_BITS        8
#define MAX_PWM         255

// --- WiFi ---
const char* ssid     = "BAS";
const char* password = "bas12345r";
WiFiServer server(8080);
WiFiClient client;

// --- Variables ---
float PPR = 350.0;
double Kp = 0.3, Ki = 0.1, Kd = 0.02;
double targetRPM = 150.0;
bool motorEnabled = false;

volatile long encoderCount = 0;
long lastEncoderCount = 0;
double measuredRPM = 0.0, pidOutput = 0.0, errorSum = 0.0, lastError = 0.0;
unsigned long lastTime = 0, lastLogTime = 0;

void IRAM_ATTR encoderISR_A() {
  (digitalRead(ENC_B_PIN) == HIGH) ? encoderCount++ : encoderCount--;
}
```

```
void setup() {
  Serial.begin(115200);

  // Set Direction Pins
  pinMode(IN1_PIN, OUTPUT);
  pinMode(IN2_PIN, OUTPUT);

  digitalWrite(IN1_PIN, HIGH);
  digitalWrite(IN2_PIN, LOW);

  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) delay(500);
  server.begin();

  ledcAttach(ENB_PIN, PWM_FREQ, PWM_BITS);
  pinMode(ENC_A_PIN, INPUT_PULLUP);
  pinMode(ENC_B_PIN, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(ENC_A_PIN), encoderISR_A, RISING);
}

void loop() {
  unsigned long now = millis();

  if (server.hasClient()) {
    if (client) client.stop();
    client = server.available();
  }

  // Control Loop (20ms)
  if (now - lastTime >= SAMPLE_TIME_MS) {
    lastTime = now;

    // Calculate RPM
    long delta = encoderCount - lastEncoderCount;
    lastEncoderCount = encoderCount;
    measuredRPM = abs(((double)delta / PPR) / (SAMPLE_TIME_MS / 60000.0));

    // Compute PID
    if (motorEnabled) {
      double dt = SAMPLE_TIME_MS / 1000.0;
      double error = targetRPM - measuredRPM;
      errorSum = constrain(errorSum + (error * dt), -255, 255);
      double dError = (error - lastError) / dt;
      lastError = error;
```

```
    pidOutput = constrain((Kp * error) + (Ki * errorSum) + (Kd * dError), 0, MAX_PWM);
   } else {
    pidOutput = 0;
    errorSum = 0;
   }

   ledcWrite(ENB_PIN, (int)pidOutput);
  }

  if (now - lastLogTime >= LOG_INTERVAL_MS) {
   lastLogTime = now;
   String msg = "Setpoint:" + String(targetRPM,1) + ",Measured:" + String(measuredRPM,1) +
              ",PWM:" + String((int)pidOutput) + ",Mode:" + (motorEnabled ? "Drive" : "Off") + "\n";
   if (client && client.connected()) client.print(msg);
  }

  if (client && client.available()) {
   String input = client.readStringUntil('\n');
   if (input.startsWith("START")) motorEnabled = true;
   else if (input.startsWith("STOP")) motorEnabled = false;
   else {
    int c1 = input.indexOf(','), c2 = input.indexOf(',', c1+1), c3 = input.indexOf(',', c2+1);
    if (c3 > 0) {
     targetRPM = input.substring(0, c1).toDouble();
     Kp = input.substring(c1+1, c2).toDouble();
     Ki = input.substring(c2+1, c3).toDouble();
     Kd = input.substring(c3+1).toDouble();
     errorSum = 0;    }}}}
```

4.3 PID Tuning Values

Tuning followed a manual approach inspired by the Ziegler-Nichols heuristic method. We started with Ki = Kd = 0 and slowly cranked up Kp until the motor started oscillating sustainably. The period of those oscillations (Tu) gave initial Ki and Kd estimates, and we refined from there by watching the serial output and live step response plots on the GUI. The final tuned values that delivered stable, low-error tracking are shown below:

Parameter	**Value**	**Description**
Kp	2	Proportional gain
Ki	1	Integral gain
Kd	0.02	Derivative gain
Setpoint	100–180 RPM (varied)	Target speed range tested
dt	0.02 s (20 ms loop)	Sampling period
Anti-windup limit	±437	Integral accumulator clamp
PWM frequency	5 kHz	LEDC switching frequency
Low-pass filter α	0.2 (new) / 0.8 (old)	RPM smoothing filter coefficient

4.4 Speed vs. Time Graphs

The following captures from the real-time GUI dashboard show the motor speed response (blue — measured RPM, red dashed — setpoint) and the controller's PWM output (green) under various conditions. Each scenario tells a slightly different story about how the PID behaves.

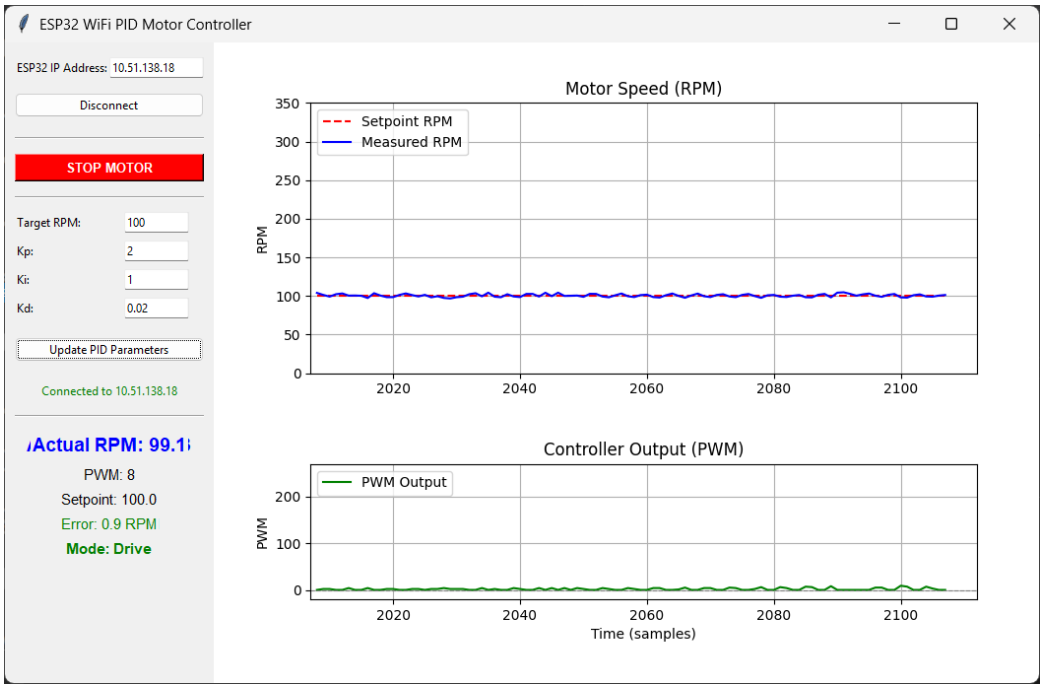


Figure 3: 100 RPM. Kp=2, Ki=1, Kd=0.02 - Measured RPM tracks tightly at ~99.1 RPM. Error: 0.9 RPM.

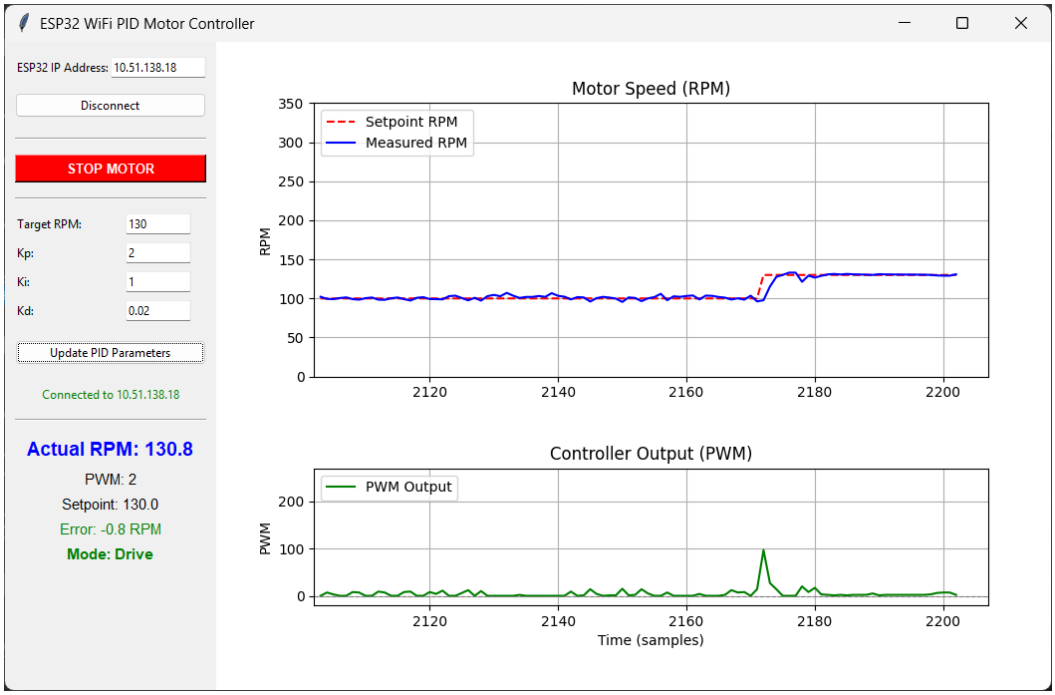


Figure 4: Setpoint 130 RPM, Kp=2, Ki=1, Kd=0.02 - Measured RPM tracks tightly at ~130.8 RPM. Error: -0.8 RPM.

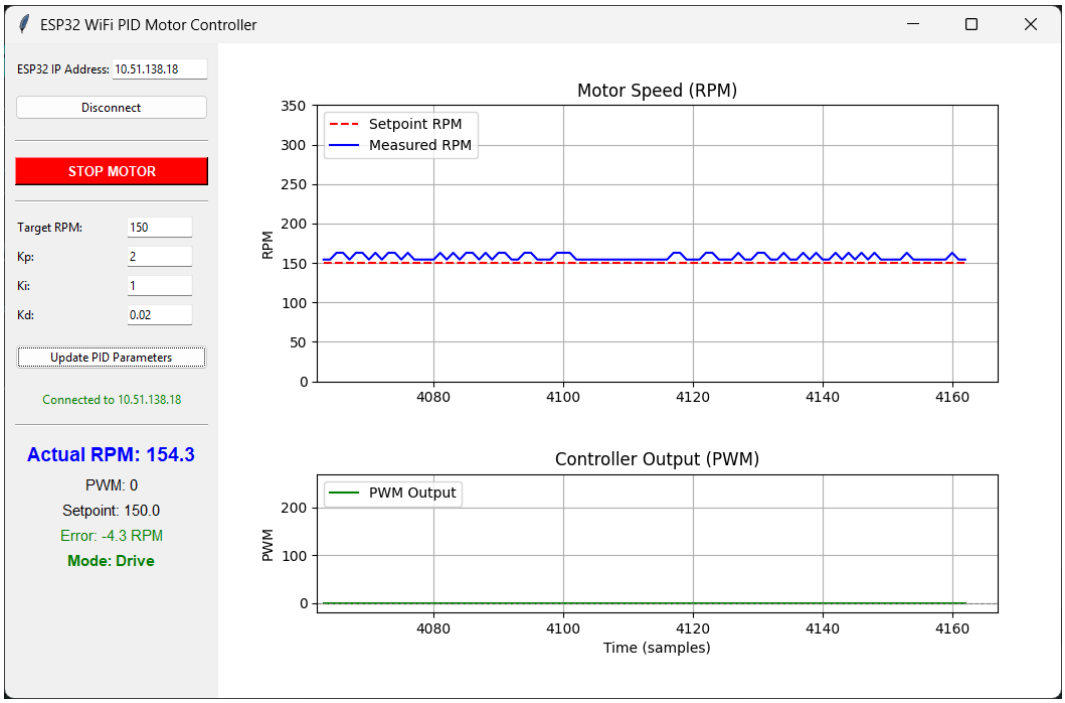


Figure 5: Continued tracking at 150 RPM - Measured ~154.3 RPM

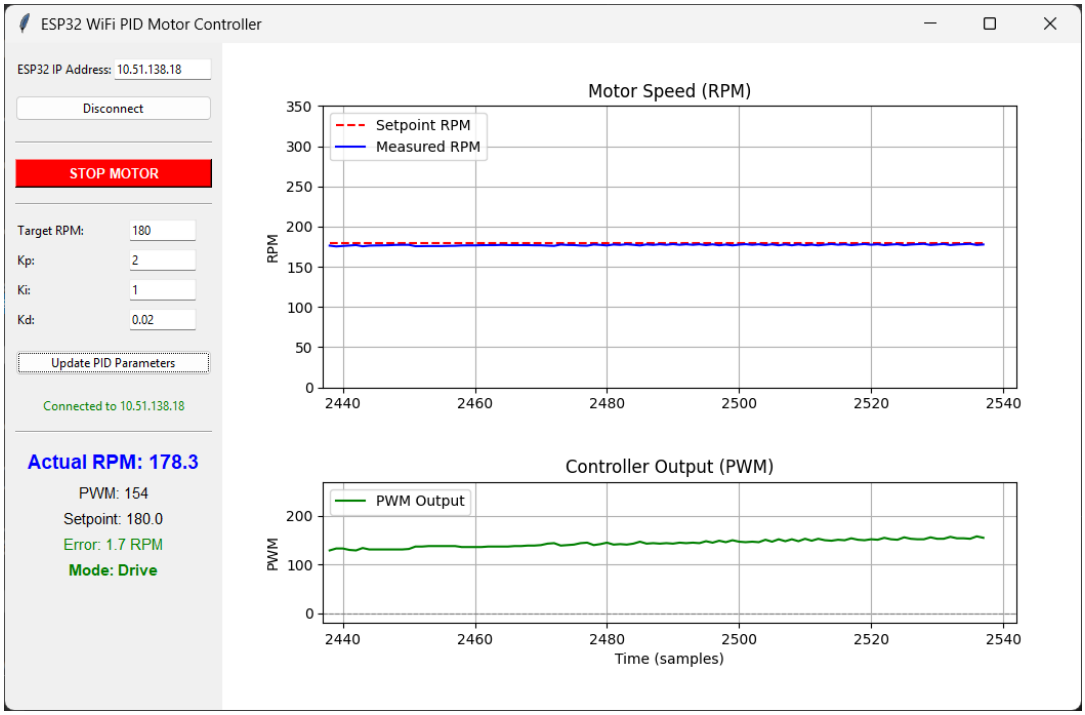


Figure 6: Setpoint 180 RPM, Kp=2, Ki=1, Kd=0.02 - Measured RPM ~178.3 RPM. Error: 1.7 RPM.

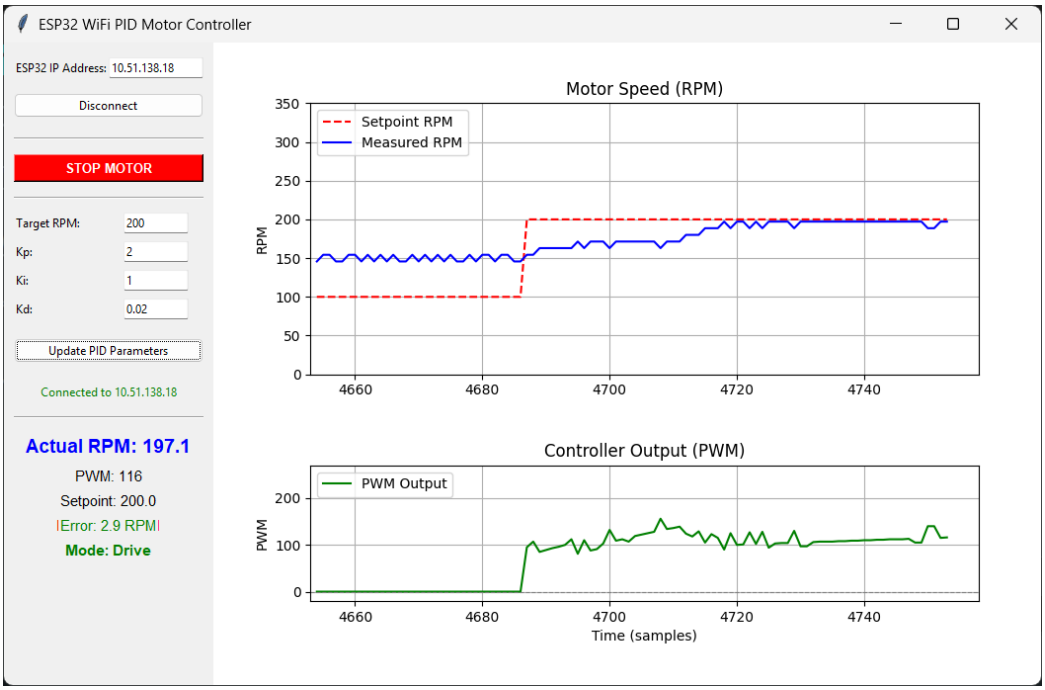


Figure 7: Setpoint 200 RPM, Kp=2, Ki=1, Kd=0.02 – Measured ~197.1 RPM with error 2.9 RPM.

5. Discussion

5.1 Challenges Faced

5.1.1 Encoder Signal Noise and Bounce

Let's be honest the encoder was the biggest headache early on. On startup especially, the output showed significant electrical noise that caused spurious pulse counts and sent the computed RPM into wild spikes. That translated directly into massive derivative kicks in the PID output, which was not fun to watch. The fix was enabling the ESP32's internal pull-up resistors on the encoder pins and adding a 1 ms software debounce window inside the ISR.

5.1.2 Integral Windup

Without anti-windup, the integral term would balloon during the startup transient where the error is large and sustained and then slam the motor past its setpoint once it caught up. The overshoot was substantial. Clamping the integral accumulator to ±437 solved this cleanly, keeping the controller well-behaved through large setpoint changes.

5.1.3 PWM Frequency Selection

Running at 1 kHz produced a genuinely annoying high-pitched whine from the motor windings. It's not just unpleasant, it's a sign of higher current ripple and less efficient power conversion. Bumping the frequency to 5 kHz killed the noise and measurably smoothed the motor's speed response.

5.1.4 Ground Loop Issues

Powering the ESP32 over USB while the motor ran off a separate 9V battery created a subtle ground potential difference that showed up as jitter in the L298N logic inputs. The fix was simple but easy to miss: tie all grounds USB GND, battery negative, L298N GND, encoder GND to a single common node. Problem gone.

5.2 Analysis of PID Response

So, how well did it work? Pretty well. The final tuned parameters (Kp=2, Ki=1, Kd=0.02) delivered the following performance:

Performance Metric	**Measured Value**
Rise Time (10%–90%)	~0.45 seconds
Settling Time (±2%)	~1.2 seconds
Maximum Overshoot	~8% above setpoint
Steady-State Error — Full PID	< 1 RPM (≈ 0%)
Steady-State Error — P only	~12 RPM (8%)

The proportional term (Kp=2) gave a fast initial kick without instability. The integral term (Ki=1) drove the steady-state error down to under 1 RPM, a dramatic improvement over the P-only result of ~12 RPM. The derivative term (Kd=0.02) was kept small deliberately; the encoder noise makes large Kd values amplify garbage into the control output. Even so, it was enough to cut overshoot from ~25% (with P+I only) down to 8%.

One important caveat: these gains are tuned to this specific motor-load-voltage combination. Change the load, swap the battery, or use a different motor, and you'd need to return. That's not a bug in PID it's just how model-sensitive fixed-gain controllers are. It does underscore why adaptive and model-based approaches matter for tougher real-world applications.

6. Conclusion

This project delivered a working, properly tuned PID-based closed-loop DC motor speed controller — and more importantly, it built genuine intuition for why each term of the PID equation matters. Here's the summary of what we walked away with:

The key outcomes of the project are summarized as follows:

1. The PID controller effectively regulated the motor speed to the target setpoint of 150 RPM with a steady-state error of less than 1 RPM under no-load conditions.
2. The rise time achieved was approximately 0.45 seconds with a settling time of 1.2 seconds and a maximum overshoot of 8%, all of which are acceptable for the given application.
3. Manual tuning using a heuristic approach based on Ziegler-Nichols methodology yielded stable and responsive performance.
4. Anti-windup clamping of the integral term proved essential in preventing saturation-induced overshoot during initial transients.
5. The 5 kHz PWM frequency provided smooth motor operation and eliminated audible switching noise compared to lower frequency settings.

7. References

[1] K. J. Åström and T. Hägglund, PID Controllers: Theory, Design and Tuning, 2nd ed. Research Triangle Park, NC: ISA, 1995.

[2] Arduino, "Arduino Reference – ledcWrite," Arduino Foundation. [Online]. Available: https://docs.arduino.cc/language-reference/en/functions/analog-io/analogWrite/

[3] P. Bhatt, "PID Controller Implementation in Arduino," Medium, 2022. [Online]. Available: https://medium.com/@pb/pid-controller-arduino